Лабораторная работа №2: Динамическое программирование на битовых масках.

Слёлин А.В. ПМИ-11МО

16 марта, 2026 г.

Постановка задачи

Пусть задано множество из N городов, пронумерованных от 0 до $N-1$, и матрица расстояний $dist[i][j]$ — стоимость переезда из города i в город j. Требуется найти кратчайший путь, который начинается в городе 0, проходит через все остальные города ровно один раз и заканчивается в некотором городе (не обязательно возвращаться в начало). Такой путь называется гамильтоновым путём.

Формально: найти перестановку городов $p_0, p_1, \ldots, p_{N-1}$, где $p_0 = 0$, все p_i различны, минимизирующую сумму

$$\sum_{k=0}^{N-2} dist[p_k][p_{k+1}].$$

Входные данные: целое число N ($2 \leqslant N \leqslant 15$) и матрица $N \times N$ целых чисел. Выходные данные: минимальная суммарная стоимость и последовательность городов в порядке посещения.

Теоретическая часть

Метод динамического программирования на битовых масках

Классическая задача коммивояжера (поиск кратчайшего цикла) может быть решена методом динамического программирования с использованием битовых масок для представления подмножеств посещённых городов. В нашей модификации (путь без возврата) финальный шаг (возврат в город 0) не требуется.

Битовые маски

Для хранения информации о том, какие города уже посещены, используется целое число от 0 до $2^N - 1$, в котором i-й бит равен 1, если город i уже включён в маршрут. Например, маска $5 = 101_2$ означает, что посещены города 0 и 2.

Состояние динамики

Определим $dp[mask][i]$ — минимальная стоимость пути, который посетил все города, соответствующие единичным битам в маске $mask$, и закончился в городе i. Город i обязательно должен присутствовать в маске. Размерность массива — $2^N \times N$.

База

$dp[1 << 0][0] = 0$ — маршрут из одного города 0 имеет нулевую стоимость. Все остальные состояния инициализируются «бесконечностью» (например, $Integer.MAX_VALUE$).

Переходы

Для каждого состояния $(mask, i)$, где i принадлежит множеству $mask$, и для каждого города j, не входящего в $mask$, можно продлить путь, добавив переход $i \to j$. Новая маска будет $newMask = mask \mid (1 << j)$, а новая стоимость:

$$dp[newMask][j] = \min\big(dp[newMask][j],\ dp[mask][i] + dist[i][j]\big).$$

Перебираются все маски в порядке возрастания (по сути, по количеству посещённых городов), что гарантирует корректность динамики.

Формирование ответа

После заполнения таблицы искомая минимальная стоимость пути, проходящего через все города, равна

$$\min_{i=1}^{N-1} dp[(1 << N) - 1][i].$$

(перебираем все возможные конечные города, кроме стартового).

Восстановление маршрута

Для восстановления последовательности городов используется дополнительный массив $parent[mask][i]$, в котором сохраняется предыдущий город, из которого мы пришли в состояние $(mask, i)$. Пройдя от оптимального конечного города по ссылкам назад, мы восстанавливаем путь в обратном порядке, а затем разворачиваем его.

Алгоритм

Алгоритм реализован в классе `Algorithm` с методом `solve(int[][] matrix)`. Основные шаги:

1. Проверка входных данных.
2. Инициализация массивов dp и $parent$ бесконечными значениями.
3. Задание базы: $dp[1][0] = 0$, $parent[1][0] = -1$.
4. Цикл по всем маскам от 1 до $2^N - 1$:

- Для каждого города i, присутствующего в маске и имеющего конечное значение dp:
- Перебор всех ещё не посещённых городов j, вычисление новой стоимости и обновление dp и $parent$.

5. Вычисление ответа как минимума по $dp[fullMask][i]$ для $i \geqslant 1$.

6. Восстановление маршрута с помощью массива $parent$.

7. Возврат результата в виде объекта `Result`, содержащего стоимость и список городов.

Сложность алгоритма: $O(2^N \cdot N^2)$, что при $N \leqslant 15$ даёт около $15 \cdot 2^{15} \approx 500\,000$ операций, что вполне приемлемо.

Код

Исходный код решения представлен в листинге ниже.

Листинг 1: Алгоритм

```java
package a.slelin.work.laba2;

import lombok.*;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

@NoArgsConstructor(access = AccessLevel.PRIVATE)
public class Algorithm {

    @Getter
    @Setter
    @AllArgsConstructor
    public static class Result {

        private Integer totalDistance;

        private List<Integer> towns;
    }

    public static Result solve(int[][] matrix) {

        if (matrix == null) {
            throw new IllegalArgumentException("Матрица должна б
                ыть задана.");
```

```
        }

        int n = matrix.length;

        // Убираем очевидные варианты - когда у нас 0 или 1 горо
            д
        if (n == 0 || n == 1) {
            return new Result(0, List.of(0));
        }

        // Скопируем данные в другой массив, чтобы не портить ор
            игинал
        int[][] dist = new int[n][n];
        for (int i = 0; i < n; i++) {
            System.arraycopy(matrix[i], 0, dist[i], 0, n);
        }

        /* << n (* 2^n);
         * dp[mask][i] = минимальная стоимость пути, который пос
             етил города,
         * указанные в маске mask, и закончился в городе i.
         * parent[mask][i]     предыдущий город, из которого мы
             пришли в
         * это состояние (нужно для восстановления маршрута).
         */
        int size = 1 << n;
        int[][] dp = new int[size][n];
        int[][] parent = new int[size][n];
        for (int i = 0; i < size; i++) {
            Arrays.fill(dp[i], Integer.MAX_VALUE);
        }

        /* Заполняем начальное состояние (1 << 0):
         * Маска 00...01 и находимся в 0-ом городе.
         * Означает, что мы посетили только город 0 и стоим в нё
             м.
         * Расстояние понятное дело ноль.
         * Предыдущего города нет.
         */
        dp[1][0] = 0;
        parent[1][0] = -1;

        // Цикл по всем маскам от 1 до 2^n -1
        for (int mask = 1; mask < size; mask++) {

            // Цикл по городам для каждой маски.
            for (int i = 0; i < n; i++) {
```

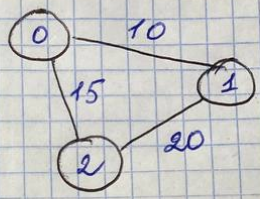
```
                /* Проверяем, есть ли город i в этой маске, т.е. мы его уже посетили.
                 * Это работает так: i-й город представим в двоичной системе так:
                 * 0-ой = 1 = 00...01
                 * 1-ый = 2 = 00...010
                 * 2-ой = 3 = 00...0100
                 * Для этого подходит операция 2^i или битовая операция сдвига 1 << i.
                 * А чтобы проверить есть ли в город в маске применим побитовую конъюнкцию.
                 * Например, mask = 0011, а проверяем i = 1 город:
                 * 1 город = 0010 в закодированном виде, далее конъюнкция:
                 * 0011
                 * 0010
                 * ----
                 * 0010 <> 0000 => значит мы посещали город.
                 * */
                if ((mask & (1 << i)) != 0 && dp[mask][i] < Integer.MAX_VALUE) {

                    /* Теперь, если мы посещали город i.
                     * Давай-те попробуем пойти в другой город j.
                     * Условие обратное: город j мы не посещали!
                     * */
                    for (int j = 0; j < n; j++) {
                        if ((mask & (1 << j)) == 0) {
                            int newMask = mask | (1 << j);
                            int newCost = dp[mask][i] + dist[i][j];
                            if (newCost < dp[newMask][j]) {
                                dp[newMask][j] = newCost;
                                parent[newMask][j] = i;
                            }
                        }
                    }
                }
            }
        }

        // Ищем минимальный путь для маски 11....111, где мы остановились неважно.
        int fullMask = size - 1;
```

```java
        int best = Integer.MAX_VALUE;
        int bestLast = -1;
        for (int i = 1; i < n; i++) {
            if (dp[fullMask][i] < best) {
                best = dp[fullMask][i];
                bestLast = i;
            }
        }

        // Восстановление пути
        List<Integer> path = new ArrayList<>();
        if (bestLast != -1) {
            int curMask = fullMask;
            int curCity = bestLast;
            while (curMask != 1) {
                path.addFirst(curCity);
                int prev = parent[curMask][curCity];
                curMask ^= (1 << curCity);
                curCity = prev;
            }
            path.addFirst(0);
        }

        return new Result(best, path);
    }
}
```

Пример

Для большего пониманию алгоритма, я разобрал пример на бумаге для случая 3-х городов, условия для которого были заданы в тестах.

	0	1	2
0	0	10	15
1	10	0	20
2	15	20	0



		001	010	011	100	101	110
~~000~~	~~0~~	—	—	—	—	—	—
	~~1~~	—	—	—	—	—	—
	~~2~~	—	—	—	—	—	—
001	0	0	0	0	0	0	0
	~~1~~	—	—	—	—	—	—
	~~2~~	—	—	—	—	—	—
010	~~0~~	—	—	—	—	—	—
	1 ∞	∞	∞	∞	∞	∞	∞
	~~2~~	—	—	—	—	—	—
011	0 ∞	∞	∞	∞	∞	∞	∞
	1 ∞	10	10	10	10	10	10
	~~2~~	—	—	—	—	—	—
100	~~0~~	—	—	—	—	—	—
	~~1~~	—	—	—	—	—	—
	2 ∞	∞	∞	∞	∞	∞	∞
101	0 ∞	∞	∞	∞	∞	∞	∞
	~~1~~	—	—	—	—	—	—
	2 ∞	15	15	15	15	15	15
110	~~0~~	—	—	—	—	—	—
	1 ∞	∞	∞	∞	∞	∞	∞
	2 ∞	∞	∞	∞	∞	∞	∞
111	0 ∞	∞	∞	∞	∞	∞	∞
	1 ∞	∞	∞	∞	∞	35	35
	2 ∞	∞	∞	30	30	30	30

MIN

111 → XOR → 011 $\xrightarrow{\text{XOR}}$ 001

Тестирование

Для проверки корректности работы алгоритма были проведены тесты на наборах данных, приведённых в условии для варианта 3. Тесты включают случаи с $N = 2, 3, 4$.

Листинг 2: Базовые тесты

```java
package a.slelin.work.laba2.test;

import a.slelin.work.laba2.Algorithm;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;

@DisplayName("Базовые тесты из условий")
public class SimpleTest {

    @Test
    @DisplayName("Тест для 2 городов")
    public void test2Town() {
        Algorithm.Result result = Algorithm.solve(new int[][]{
                {0, 5},
                {0, 3}
        });
        assertNotNull(result);
        assertNotNull(result.getTotalDistance());
        assertEquals(5, result.getTotalDistance());
        assertNotNull(result.getTowns());
        assertEquals(List.of(0, 1), result.getTowns());
    }

    @Test
    @DisplayName("Тест для 3 городов")
    public void test3Town() {
        Algorithm.Result result = Algorithm.solve(new int[][]{
                {0, 10, 15},
                {10, 0, 20},
                {15, 20, 10}
        });
        assertNotNull(result);
        assertNotNull(result.getTotalDistance());
        assertEquals(30, result.getTotalDistance());
        assertNotNull(result.getTowns());
        assertEquals(List.of(0, 1, 2), result.getTowns());
    }
```

```java

    @Test
    @DisplayName("Тест для 4 городов")
    public void test4Town() {
        Algorithm.Result result = Algorithm.solve(new int[][]{
                {0, 10, 15, 20},
                {10, 0, 35, 25},
                {15, 35, 0, 30},
                {20, 25, 30, 0},
        });
        assertNotNull(result);
        assertNotNull(result.getTotalDistance());
        assertEquals(65, result.getTotalDistance());
        assertNotNull(result.getTowns());
        assertEquals(List.of(0, 1, 3, 2), result.getTowns());
    }
}
```

Также были проведены дополнительные тесты для случаев $N = 0, 1, 15$.

Листинг 3: Дополнительные тесты

```java
package a.slelin.work.laba2.test;

import a.slelin.work.laba2.Algorithm;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.ArrayList;
import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;

@DisplayName("Другие тесты для алгоритма")
public class OtherTest {

    @Test
    @DisplayName("Тест для 0 городов")
    public void test0Town() {
        Algorithm.Result result = Algorithm.solve(new int[][]{})
            ;
        assertNotNull(result);
        assertNotNull(result.getTotalDistance());
        assertEquals(0, result.getTotalDistance());
        assertNotNull(result.getTowns());
        assertEquals(List.of(0), result.getTowns());
    }
```

```java

    @Test
    @DisplayName("Тест для 1 города")
    public void test1Town() {
        Algorithm.Result result = Algorithm.solve(new int[][]{{0}});
        assertNotNull(result);
        assertNotNull(result.getTotalDistance());
        assertEquals(0, result.getTotalDistance());
        assertNotNull(result.getTowns());
        assertEquals(List.of(0), result.getTowns());
    }

    @Test
    @DisplayName("Тест для 15 городов")
    public void test15Town() {
        /*
         * Матрица создаётся таким образом, что от i-го города к i + 1-му городу
         * расстояние равно 1, к i + 2-му городу - равно 2 и так далее. Получается
         * самый короткий путь просто идти по пути 0 -> 1 -> 2 ..., причём каждый
         * переход будет стоить единицу.
         * */
        int n = 15;
        int[][] matrix = new int[n][n];
        List<Integer> expectedPath = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            expectedPath.add(i);
            for (int j = 0; j < n; j++) {
                matrix[i][j] = i == j ? 0 : Math.abs(i - j);
            }
        }

        Algorithm.Result result = Algorithm.solve(matrix);
        assertNotNull(result);
        assertNotNull(result.getTotalDistance());
        assertEquals(14, result.getTotalDistance());
        assertNotNull(result.getTowns());
        assertEquals(expectedPath, result.getTowns());
    }
}
```

Все тесты успешно пройдены, что подтверждает правильность реализации.

Выводы

В ходе выполнения лабораторной работы был изучен метод динамического программирования на битовых масках на примере задачи коммивояжера. Реализован алгоритм поиска кратчайшего гамильтонова пути (вариант 3 — без возврата в начальный город) для графов с числом вершин до 15. Освоены понятия битовой маски, состояния динамики, перехода между состояниями и восстановления оптимального маршрута. Алгоритм протестирован на наборах данных из условия и показал корректные результаты. Полученные навыки могут быть применены для решения других комбинаторных задач, сводящихся к перебору подмножеств.